

In OOP, by inserting virtual, you can decide whether a method can be modified or not.

Not only that, the original method still exists whether or not you have modified it, and any changes you make only apply to that particular sub-class.

The System Verilog code for 'is also' functionality is:

```
class DbTransT extends TransBaseT;
    virtual function processTrans();
        super.processTrans();
        //Local functionality goes here
    endfunction
endclass

class IdbTransT extends DbTransT;
    function processTrans();
        super.processTrans();
        //Local functionality goes here
```

```
endfunction
endclass

DbTransT e = new;
IdbTransT t1 = new;
TransBaseT p;
TransBaseT p2 = new;

p2.processTrans(); //Original
Version
e.processTrans(); //DbTransT Version
t1.processTrans(); //IDB Version

p = e;

p.processTrans(); //DbTrans Version
```

OOP provides the flexibility of keeping the methods in both the base and all extended classes. By using the power of polymorphism, you can choose and pick which

function to call.

The bottomline

As discussed in the article, System Verilog allows you to do all the things that AOP does—it's simply a matter of understanding it. It provides painless introduction to OOP, allowing you to use as little or much of OOP as you feel comfortable with, so understanding it is not difficult for verification engineers.

Comparing AOP with OOP, it is very obvious that the main difference is in organisation of the code. In AOP you organise your code by functionality, while in OOP you organise your code by classes and objects. ●

The author is a project leader at elfnchips with more than seven years of experience in Chip/ASIC division

Want to have an edge over others?

Master LINUX



Read LINUX For You

For more info, log on to www.linuxforu.com

ASIA'S FIRST
LINUX MAGAZINE

